

Proyecto Bases De Datos: Chocolate Doom



Presentado a:
Andres Oswaldo Calderón

Presentado por:
Juan José Castillo Escobar
Laura Gabriela Rueda
Nicolas Espinosa
Alejandro Mendez

1. Introducción:

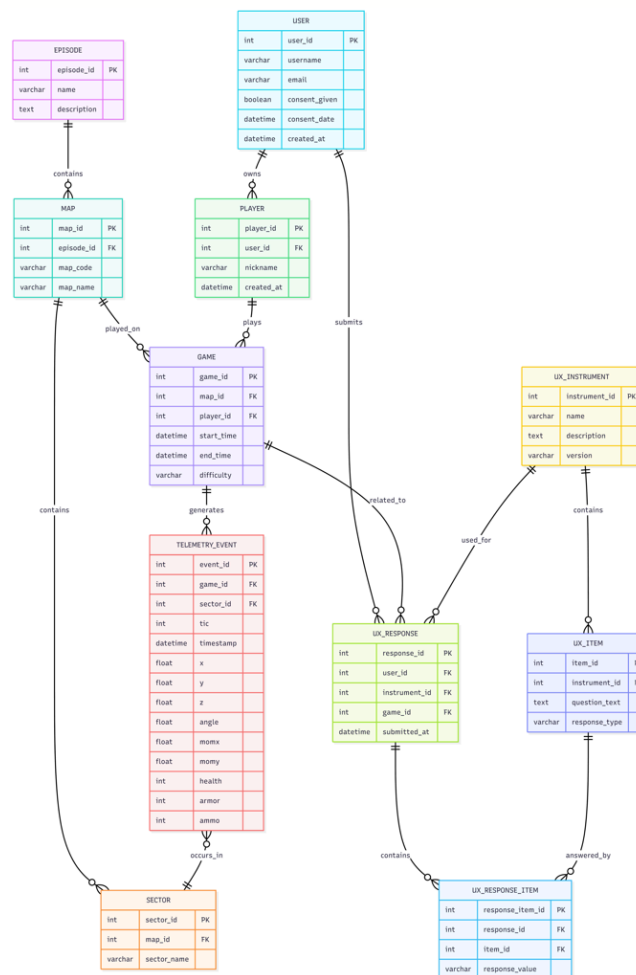
En el presente documento se presenta el contenido desarrollado en el proyecto que fue necesario para el desarrollo del mismo, esto con el fin de exponer de la mejor manera los conceptos vistos en el curso, que concluyen con este proyecto.

Primero se podrá apreciar todo el proceso de análisis del problema con su modelado y diagrama Entidad-Relación, del mismo modo el diagrama DDL, que nos muestra la estructura de modelado de datos para la creación de la base de datos.

En segunda instancia se mostrará el proceso de ETL, y toda la implementación de los datos en archivos TSV para incorporarlas a la base de datos y poblarla.

Finalmente se mostrará todo el análisis de la base de datos, con el uso de queries para definir los comportamientos de los jugadores, items y demás aspectos de las partidas.

2. Diagrama E-R:



El diagrama es un modelo entidad-relación que integra la telemetría de un videojuego tipo Doom (usuarios, jugadores, episodios, mapas, sectores, partidas y eventos de telemetría que registran posición, momento, salud, armadura y munición por *tic*) con la medición de experiencia de usuario mediante instrumentos, ítems y respuestas (UX_INSTRUMENT, UX_ITEM, UX_RESPONSE,

UX_RESPONSE_ITEM), de modo que el comportamiento objetivo en el juego pueda correlacionarse con la percepción subjetiva del usuario por partida; conforme a las correcciones de sustentación, se debe eliminar todo lo relativo al multijugador (simplificando el modelo a un solo jugador por partida), quitar la relación directa entre MAP y la telemetría (los eventos deben asociarse a la partida y al sector, no al mapa), reflejar que el jugador llega o se vincula a la partida en lugar de que esta dependa de relaciones redundantes, y aplicar en general estos ajustes de cardinalidad y consistencia para depurar el modelo ER.

3. DDL/Restricciones:

Fragmentos clave del DDL implementados con el fin de obtener diversos objetivos:

- a) Para garantizar que una partida siempre se asocie a un mapa y a un jugador válido.

```
CREATE TABLE game (  
  game_id SERIAL PRIMARY KEY,  
  map_id INT NOT NULL,  
  player_id INT NOT NULL,
```

```
  CONSTRAINT fk_game_map  
    FOREIGN KEY (map_id)  
    REFERENCES map(map_id)  
    ON DELETE RESTRICT,
```

```
  CONSTRAINT fk_game_player  
    FOREIGN KEY (player_id)  
    REFERENCES player(player_id)  
    ON DELETE RESTRICT
```

```
);
```

- b) Para evitar incorporar niveles de dificultad no válidos.

```
CONSTRAINT chk_game_difficulty  
CHECK (  
  difficulty IN ('easy','medium','hard','nightmare')  
)
```

- c) Para garantizar que un jugador no tenga registrados dos eventos para un mismo tic dentro de una misma partida

```
CONSTRAINT uq_telemetry_event  
UNIQUE (  
  game_id,  
  tic,  
  player_id  
)
```

- d) Para mantener los valores de armadura y salud entre el dominio
- ```
CONSTRAINT chk_health
CHECK (
 health IS NULL
 OR (health BETWEEN 0 AND 200)
),

CONSTRAINT chk_armor
CHECK (
 armor IS NULL
 OR (armor BETWEEN 0 AND 200)
)
```
- e) Limitar los instrumentos a guess bangs y pens
- ```
CONSTRAINT chk_instrument_name
CHECK (
    name IN ('PENS', 'GUESS', 'BANGS')
)
```
- f) Recepción de datos en crudo antes de cargarlos al esquema general
- ```
CREATE TABLE telemetry_staging (
 raw_id BIGSERIAL PRIMARY KEY,
 game_id TEXT,
 player_id TEXT,
 sector_id TEXT,
 tic TEXT,
 ts TEXT,
 x TEXT,
 y TEXT,
 z TEXT,
 loaded_at TIMESTAMP DEFAULT NOW()
);
```
- g) Para optimizar las consultas de trayectorias uso de indice
- ```
CREATE INDEX idx_te_game_player_tic
ON telemetry_event (
    game_id,
    player_id,
    tic
);
```

4. ETL descripción:

Extract (Extracción): En esta primera etapa el script toma la telemetría cruda de los archivos .tsv que se generaron al jugar DOOM con Chocolate Doom, procesando un archivo por cada jugador. Antes de cargar cada archivo se limpia la tabla intermedia (staging) para que no queden datos del jugador anterior, y luego se importa el archivo con el comando COPY, usando el tabulador como separador. Todos los datos entran tal cual, como texto, sin revisarlos todavía. Así la tabla de staging funciona como un espacio temporal donde los datos quedan guardados antes de procesarlos.

Transform (Transformación): Aquí se limpian y se ordenan los datos que quedaron en el staging. Se le asigna a cada jugador su número de partida (game_id), se revisa fila por fila para marcar cuáles sirven y cuáles no, y se anota el motivo cuando una fila tiene un error (por ejemplo, coordenadas mal escritas o partida sin asignar). También se calcula el sector a partir de las coordenadas del mapa y se convierten los datos de texto a su tipo correcto: números enteros, números decimales y fechas. De esta forma los datos quedan corregidos y listos para guardarse.

Load (Carga): En esta última etapa solo pasan a las tablas finales las filas que quedaron marcadas como válidas. Primero se guardan los sectores que no existan todavía, evitando repetirlos, y después se guardan los eventos de telemetría en la tabla telemetry_event, conservando el map_id para saber a qué mapa o nivel pertenece cada evento, ya que cada nivel tiene su propio mapa. Además, antes de insertar se revisa que el evento no esté ya guardado (según la partida y el tic), de modo que si se vuelve a correr el script no se dupliquen los datos.

5. Queries + resultados

Q1: Duraciones promedio de sesiones de juego por mapa, calcula cuantos segundos duro en promedio cada partida, agrupado por mapa. Usa los time stamps de inicio a fin de game.

	map_code character varying (20)	map_name character varying (100)	total_sesiones bigint	duracion_promedio_seg numeric	duracion_promedio_min numeric
1	E1M3	Toxin Refinery	6	13199.00	219.98
2	E1M2	Nuclear Plant	6	1199.00	19.98
3	E1M1	Hangar	6	1199.00	19.98

Q3: Trayectoria más corta y más larga por jugador

	jugador character varying (100)	trayectoria_mas_corta numeric	trayectoria_mas_larga numeric	trayectoria_promedio numeric
1	sebastian	118060.97	121852.95	119956.96
2	cristian	110609.77	111526.99	111068.38
3	nicolas	100167.37	101729.41	100948.39
4	alejandro	91497.33	92644.41	92070.87
5	juanjose	82473.54	82509.90	82491.72
6	laura	73957.67	74338.14	74147.90

Q5: Sector mas visitado (Hotspot) por episodio y mapa, cuenta cuantos tcs registro cada sector en cada mapa, y devuelve el top 3 sectores mas visitados.

	episodio character varying (100)	map_code character varying (20)	sector character varying (100)	total_tics bigint	ranking bigint
1	Knee-Deep in the Dead	E1M1	Grid (-3,-16)	39	1
2	Knee-Deep in the Dead	E1M1	Grid (3,5)	36	2
3	Knee-Deep in the Dead	E1M1	Grid (10,-25)	35	3
4	Knee-Deep in the Dead	E1M3	Grid (-4,18)	49	1
5	Knee-Deep in the Dead	E1M3	Grid (-3,19)	43	2
6	Knee-Deep in the Dead	E1M3	Grid (-7,6)	41	3

Q6: Tics de co-presencia en el mismo sector entre jugadores, cuenta cuantos tics distintos dos jugadores compartieron el mismo sector dentro de la misma partida

	jugador_1 character varying (100)	jugador_2 character varying (100)	map_code character varying (20)	tics_juntos bigint
1	nicolas	sebastian	E1M1	6
2	laura	cristian	E1M3	5
3	alejandro	cristian	E1M1	4
4	laura	sebastian	E1M1	3
5	juanjose	cristian	E1M1	2

Q7: A grupa a los jugadores por episodio y les asigna el número 1 al que tenga la distancia más baja.

	episodio character varying (100)	jugador_ruta_corta character varying (100)	distancia_recorrida numeric	puntaje_ux_promedio numeric
1	Knee-Deep in the Dead	laura	148295.80	3.94

Q8: Distancia total y velocidad promedio por jugador, Suma distancias Euclidianas consecutivas en todos los games de un jugador y divide por el total de tics para obtener la velocidad promedio

	jugador character varying (100)	total_games bigint	total_tics bigint	distancia_total_doom_units numeric	velocidad_promedio_u_por_tic numeric
1	sebastian	2	2396	239913.93	100.1310
2	cristian	2	2396	222136.77	92.7115
3	nicolas	2	2396	201896.78	84.2641
4	alejandro	2	2396	184141.73	76.8538
5	juanjose	2	2396	164983.44	68.8579
6	laura	2	2396	148295.80	61.8931

6. Evaluación de índices

Con el objetivo de mejorar el rendimiento de las consultas más frecuentes sobre los datos de telemetría, se implementaron varios índices especializados en la base de datos. Estos índices permiten reducir el tiempo necesario para localizar registros, evitando en muchos casos recorridos completos de las tablas.

El índice `idx_te_game_tic`, definido sobre las columnas (`game_id`, `tic`) de la tabla `telemetry_event`, está diseñado para optimizar consultas que filtran eventos de telemetría pertenecientes a una partida específica y que requieren ordenar o recorrer los datos según el instante de tiempo registrado. Este índice facilita la reconstrucción de secuencias de eventos dentro de una sesión de juego y mejora significativamente el acceso a registros asociados a una partida concreta.

El índice `idx_te_map_sector`, construido sobre las columnas (`map_id`, `sector_id`), mejora las consultas que analizan la distribución de eventos en diferentes zonas de un mapa. Gracias a este índice, PostgreSQL puede localizar rápidamente los registros asociados a un mapa y sector específicos, reduciendo el tiempo de búsqueda y favoreciendo análisis espaciales relacionados con la actividad de los jugadores.

Por su parte, el índice `idx_te_pos` utiliza una estructura GiST sobre el punto geométrico formado por las coordenadas (`x`, `y`). Este índice está orientado a consultas espaciales, permitiendo realizar búsquedas eficientes de eventos dentro de áreas determinadas del mapa. Su implementación resulta especialmente útil para la generación de mapas de calor, análisis de movimiento y detección de patrones espaciales dentro del entorno de juego.

Para evaluar la efectividad de los índices, se ejecutaron las consultas correspondientes antes y después de su creación utilizando la herramienta `EXPLAIN ANALYZE` de PostgreSQL. Los resultados obtenidos muestran una mejora significativa en el rendimiento de las consultas evaluadas.

Los tiempos de ejecución registrados fueron los siguientes:

Query	Sin índice	Con índice	% Mejora
Q1	0.883	0.774	12.3%
Q2	1.839	0.104	94.3%
Q3	2.863	0.254	91.1%

El análisis de rendimiento demuestra el impacto positivo de la indexación en la ejecución de consultas. Si bien la Consulta 1 mostró una mejora limitada debido a su ya bajo costo de ejecución, las Consultas 2 y 3 se beneficiaron considerablemente del uso de índices, reduciendo los tiempos de ejecución de 1,839 ms a 0,104 ms y de 2,863 ms a 0,254 ms, respectivamente. Estos resultados confirman que los índices seleccionados soportan eficazmente los patrones de acceso más exigentes de la base de datos, minimizando los escaneos secuenciales y mejorando la eficiencia de la recuperación de datos. Por lo tanto, la estrategia de indexación contribuye a un mejor rendimiento, especialmente a medida que aumenta el volumen de datos de telemetría.

7. Diccionario de datos:

USER

Almacena la información del voluntario participante en la investigación. Incluye control de consentimiento informado.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
user_id	SERIAL	No	✓		Identificador único auto incremental del usuario.
username	VARCHAR(64)	No			Nombre de usuario único en el sistema.
email	VARCHAR(128)	Sí			Correo electrónico del usuario. Puede ser nulo.
consent_given	BOOLEAN	No			Indica si el usuario firmó el consentimiento informado. Por defecto FALSE.
consent_date	TIMESTAMP	Sí			Fecha y hora en que se registró el consentimiento. Nulo si aún no ha consentido.
created_at	TIMESTAMP	No			Fecha y hora de registro del usuario en el sistema. Por defecto NOW().

PLAYER

Identidad del jugador dentro del juego. Puede estar vinculada a un USER (voluntario) o ser anónima si user_id es nulo.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
player_id	SERIAL	No	✓		Identificador único autoincremental del jugador.
user_id	INT	Sí		✓	Referencia a USER(user_id). Nulo si el jugador es anónimo. ON DELETE SET NULL.
nickname	VARCHAR(64)	No			Alias del jugador en el juego.
created_at	TIMESTAMP	No			Fecha y hora de creación del perfil de jugador.

EPISODE

Nivel superior de la jerarquía de estructura del juego. Agrupa un conjunto de mapas.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
episode_id	SERIAL	No	✓		Identificador único autoincremental del episodio.

name	VARCHAR(64)	No			Nombre del episodio (ej. 'Knee-Deep in the Dead').
description	TEXT	Sí			Descripción opcional del episodio.

MAP

Mapa de juego perteneciente a un episodio. Cada partida se desarrolla en un único mapa.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
map_id	SERIAL	No	✓		Identificador único autoincremental del mapa.
episode_id	INT	No		✓	Referencia a EPISODE(episode_id). ON DELETE CASCADE.
map_code	VARCHAR(16)	No			Código del mapa según convención del juego (ej. 'E1M1').
map_name	VARCHAR(64)	No			Nombre descriptivo del mapa.

SECTOR

Subdivisión espacial de un mapa. Representa celdas de 250×250 unidades para análisis de hotspots y co-presencia.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
sector_id	SERIAL	No	✓		Identificador único autoincremental del sector.
map_id	INT	No		✓	Referencia a MAP(map_id). ON DELETE CASCADE.
sector_name	VARCHAR(64)	Sí			Nombre o código identificador del sector. Puede ser nulo.

GAME

Sesión de juego individual. Registra configuración, mapa y tiempos de inicio y fin de la partida.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
game_id	SERIAL	No	✓		Identificador único autoincremental de la partida.
map_id	INT	No		✓	Referencia a MAP(map_id). ON DELETE RESTRICT.
start_time	TIMESTAMP	No			Marca de tiempo de inicio de la sesión de juego.
end_time	TIMESTAMP	Sí			Marca de tiempo de fin. Nulo si la sesión está en curso.

difficulty	VARCHAR(16))	No			Nivel de dificultad. CHECK: ('easy','medium','hard','nightmare').
-------------------	------------------	----	--	--	---

TELEMETRY_EVENT

Registro atómico del estado del jugador por tic (1/35 segundo). Es la tabla de mayor volumen del sistema. Constraint UNIQUE sobre (game_id, tic, player_id) para deduplicación ETL.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
event_id	BIGSERIAL	No	✓		Identificador único autoincremental. BIGINT por alto volumen de registros.
game_id	INT	No		✓	Referencia a GAME(game_id). ON DELETE CASCADE.
player_id	INT	No		✓	Referencia a PLAYER(player_id). ON DELETE CASCADE.
sector_id	INT	Sí		✓	Referencia a SECTOR(sector_id). Nulo si no se puede determinar el sector.
tic	INT	No			Número de tic emitido por el motor del juego (unidad temporal discreta).
timestamp	TIMESTAMP	No			Marca de tiempo absoluta del evento de telemetría.
x	FLOAT	No			Posición del jugador en el eje X del mapa.
y	FLOAT	No			Posición del jugador en el eje Y del mapa.
z	FLOAT	No			Posición del jugador en el eje Z (altura).
angle	FLOAT	No			Ángulo de orientación del jugador en el momento del tic.
momx	FLOAT	No			Componente del vector de momentum en el eje X.
momy	FLOAT	No			Componente del vector de momentum en el eje Y.
health	INT	Sí			Puntos de salud del jugador en el momento del tic (0–200).
armor	INT	Sí			Puntos de armadura del jugador en el momento del tic (0–200).
ammo	INT	Sí			Munición disponible del jugador en el momento del tic.

UX_INSTRUMENT

Define un instrumento de evaluación UX predefinido. Los valores permitidos son PENS, GUESS y BANGS.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
instrument_id	SERIAL	No	✓		Identificador único autoincremental del instrumento.
name	VARCHAR(32)	No			Nombre del instrumento. CHECK: ('PENS','GUESS','BANGS'). UNIQUE.
description	TEXT	Sí			Descripción del propósito y estructura del instrumento.
version	VARCHAR(16)	No			Versión del instrumento utilizada en la investigación.

UX_ITEM

Pregunta individual perteneciente a un instrumento UX. Define el texto y el tipo de respuesta esperada.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
item_id	SERIAL	No	✓		Identificador único autoincremental del ítem.
instrument_id	INT	No		✓	Referencia a UX_INSTRUMENT(instrument_id). ON DELETE CASCADE.
question_text	TEXT	No			Texto completo de la pregunta del instrumento.
response_type	VARCHAR(32)	No			Tipo de respuesta esperada. CHECK: ('likert','open','binary').

UX_RESPONSE

Encabezado de la respuesta de un usuario a un instrumento UX completo, vinculada a una sesión de juego.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
response_id	SERIAL	No	✓		Identificador único autoincremental de la respuesta.
user_id	INT	No		✓	Referencia a USER(user_id). ON DELETE CASCADE.
instrument_id	INT	No		✓	Referencia a UX_INSTRUMENT(instrument_id).
game_id	INT	No		✓	Referencia a GAME(game_id). Vincula la respuesta a una sesión. ON DELETE CASCADE.
submitted_at	TIMESTAMP	No			Fecha y hora de envío del instrumento. Por defecto NOW().

UX_RESPONSE_ITEM

Respuesta atómica a un ítem específico dentro de una UX_RESPONSE. Permite análisis a nivel de pregunta individual.

Atributo	Tipo de Dato	Nulo	PK	FK	Descripción
response_item_id	SERIAL	No	✓		Identificador único autoincremental del ítem respondido.
response_id	INT	No		✓	Referencia a UX_RESPONSE(response_id). ON DELETE CASCADE.
item_id	INT	No		✓	Referencia a UX_ITEM(item_id).
response_value	VARCHAR(256)	No			Valor de la respuesta (número Likert, texto libre o 'yes'/'no').

8. Conclusiones

El desarrollo de este proyecto permitió consolidar los conceptos fundamentales del diseño, implementación y análisis de bases de datos relacionales a través de un caso de uso concreto y con datos reales: la telemetría de partidas de Chocolate Doom.

En cuanto al modelado, el diagrama Entidad-Relación demostró ser una herramienta esencial para estructurar un dominio complejo que combina datos de juego (episodios, mapas, sectores, eventos por tic) con datos de experiencia de usuario (instrumentos UX, ítems y respuestas). Las correcciones aplicadas durante la sustentación —eliminar relaciones redundantes, simplificar el modelo a un jugador por partida y desacoplar el mapa de los eventos de telemetría— reflejan que un buen modelo no se obtiene en el primer intento, sino mediante revisión iterativa.

El proceso ETL evidenció la importancia de contar con una tabla de staging como buffer intermedio antes de cargar datos al esquema final. Esta capa permitió detectar y registrar errores (coordenadas inválidas, game_id no asignados) sin comprometer la integridad de las tablas principales, y facilitó la ejecución repetible del proceso sin duplicar registros gracias al control por (game_id, tic).

El análisis mediante queries reveló patrones de comportamiento significativos entre los jugadores. Sebastian y Cristian presentaron las trayectorias y velocidades más altas, lo que puede indicar mayor nivel de actividad o exploración, mientras que Laura registró los valores más bajos en ambas métricas. Asimismo, la co-presencia en sectores compartidos (Q6) abre la posibilidad de estudiar dinámicas de interacción espacial incluso en partidas que se corren de forma independiente.

Respecto a la indexación, los resultados de EXPLAIN ANALYZE confirmaron que una estrategia de índices bien diseñada tiene un impacto directo y medible en el rendimiento. Las consultas Q2 y Q3 redujeron sus tiempos en más del 90% gracias a los índices sobre (game_id, tic) y (map_id, sector_id), lo que demuestra que la indexación es especialmente valiosa en tablas de alto volumen como telemetry_event, donde los escaneos secuenciales se vuelven costosos a medida que crecen los datos.

Finalmente, la integración del instrumento BANGS permitió correlacionar el comportamiento objetivo (distancia recorrida, velocidad, sectores visitados) con la percepción subjetiva del jugador, enriqueciendo el análisis más allá de los datos de telemetría puros. Este enfoque mixto representa una contribución metodológica relevante para investigaciones futuras en el área de experiencia de usuario en videojuegos.